

Question 1 – Conceptual: Technical Debt Diagnosis

1. Identify the hidden-technical-debt category for each case:

- (a) **Entanglement (CACE):** Changing the “estimated delivery time” feature unexpectedly affects the unrelated “favorite restaurants” feature, showing that changes in one part of the ML system affect other parts.
- (b) **Undeclared Consumers:** The marketing dashboard team silently consumes the model’s raw output table without the ML team’s knowledge, creating an invisible dependency.
- (c) **Configuration & Glue-Code Debt:** The training pipeline consists of 14 undocumented shell scripts with no orchestration tool, creating tangled and poorly managed pipeline glue code.

2. Mitigation for (c):

we can use DVC pipelines to replace the chain of undocumented shell scripts with explicitly defined pipeline stages and dependencies. Each stage can specify its inputs, outputs, and commands, making the execution order and dependencies reproducible. For example, the workflow can be organized as data validation → preprocessing → training → evaluation. This reduces the tangled “pipeline jungle,” makes the workflow easier to understand and reproduce, and reduces undocumented manual steps.

Q2.2 – Analysis of experiments

I ran six MLP experiments with different architectures and learning rates and compared their results in MLflow. From the comparison, I obtained the following observations:

- **Best-performing run:** `mlp-h128_64-lr0.001-bs128` achieved a test accuracy of **0.9731**, test macro-F1 of **0.9730**, and validation accuracy of **0.9724**. It performed slightly better than the larger (256,128,64) architecture, showing that increasing network size did not provide a significant improvement for this MNIST dataset.
- **Overfitting:** From the `train_loss` and `val_accuracy` trends, I observed some evidence of overfitting. Training loss generally continued to decrease, while validation accuracy started to plateau or fluctuate. This indicates that the model continued improving on training data without a similar improvement in validation performance. This was more noticeable for the runs with learning rate 0.01.
- **Effect of hyperparameters:** The **learning rate had the larger effect on performance**. The runs with learning rate 0.001 generally achieved better and more stable results than the 0.01 runs. Therefore, 0.001 provided better generalization for these experiments.